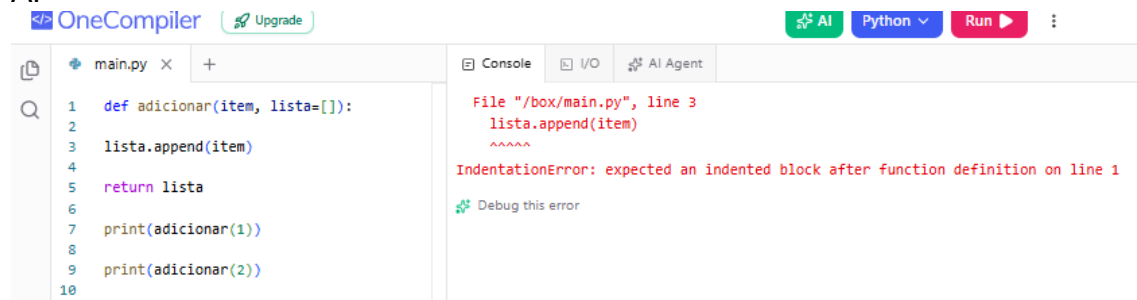


1 · Python

A:



The screenshot shows the OneCompiler Python IDE. The code in main.py is as follows:

```
1 def adicionar(item, lista=[]):
2     lista.append(item)
3
4     return lista
5
6 print(adicionar(1))
7
8 print(adicionar(2))
9
10
```

The console shows an error:

```
File "/box/main.py", line 3
    lista.append(item)
    ^^^^^
IndentationError: expected an indented block after function definition on line 1
```

B: O conceito é parâmetro com valor padrão mutável. A lista [] é criada uma única vez e reutilizada nas chamadas, fazendo com que os valores anteriores permaneçam.

2 · Java

A:



The screenshot shows the OneCompiler Java IDE. The code in Main.java is as follows:

```
1 import java.util.*;
2
3 public class Main {
4     static void zera(int[] v, int n) {
5
6         v[0] = 0;
7
8         n = 0;
9     }
10
11     int[] v = {5, 5}; int n = 5;
12
13     zera(v, n);
14
15     System.out.println(v[0] + " " + n);
16
17 }
```

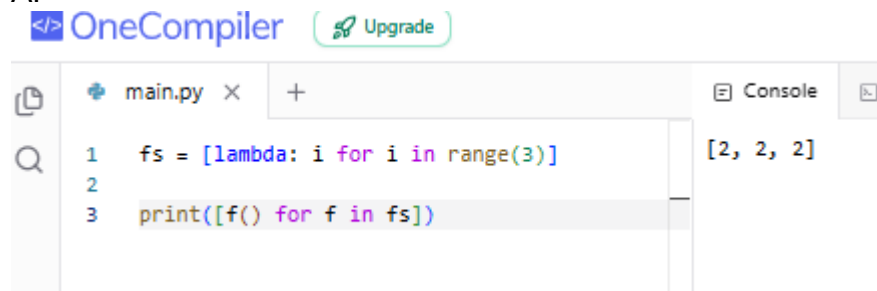
The console shows 5 errors:

```
Main.java:14: error: invalid method declaration; return type required
    zera(v, n);
    ^
Main.java:14: error: <identifier> expected
    zera(v, n);
    ^
Main.java:14: error: <identifier> expected
    zera(v, n);
    ^
Main.java:16: error: <identifier> expected
    System.out.println(v[0] + " " + n);
    ^
Main.java:16: error: ';' expected
    System.out.println(v[0] + " " + n);
    ^
5 errors
```

B: Conceito: passagem de parâmetros por valor. Em Java, a referência do array é copiada, então v[0] pode alterar o objeto original. Já n é uma cópia, então n = 0 não altera o n original.

3 • Python

A:



The screenshot shows the OneCompiler Python IDE. The editor has a tab for 'main.py'. The code in the editor is:

```
1 fs = [lambda: i for i in range(3)]
2
3 print([f() for f in fs])
```

The console on the right shows the output: [2, 2, 2].

B: Conceito: fechamento (closure). As funções lambda capturam a variável *i*, não seu valor em cada iteração. Quando são executadas, *i* já vale 2.

4 • C

A:



The screenshot shows the OneCompiler C IDE. The editor has a tab for 'NewFile1.c'. The code in the editor is:

```
1 #include <stdio.h>
2 int contador(void) {
3
4     static int n = 0;
5
6     return ++n;
7
8 }
9
10 // em main:
11
12 contador(); contador();
13
14 printf("%d\n", contador());
15 }
16
```

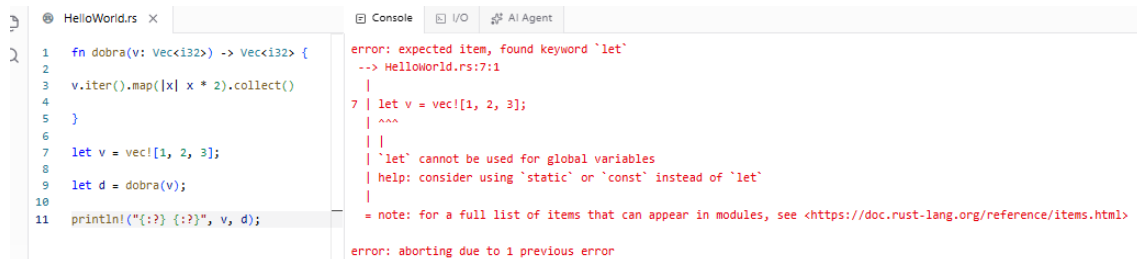
The console on the right shows several compilation errors and warnings:

- NewFile1.c:12:1: warning: data definition has no type or storage class
- NewFile1.c:12:13: warning: data definition has no type or storage class
- NewFile1.c:12:1: error: type defaults to 'int' in declaration of 'contador' [-Wimplicit-int]
- NewFile1.c:12:13: warning: data definition has no type or storage class
- NewFile1.c:12:13: error: type defaults to 'int' in declaration of 'contador' [-Wimplicit-int]
- NewFile1.c:14:8: error: expected declaration specifiers or '...' before string constant
- NewFile1.c:14:16: error: expected declaration specifiers or '...' before 'contador'
- NewFile1.c:15:1: error: expected identifier or '(' before '}' token

B: Conceito: variável estática (static). A variável *n* mantém seu valor entre as chamadas da função: $1 \rightarrow 2 \rightarrow 3$.

5 • Rust

A:



The screenshot shows a Rust IDE with a file named 'HelloWorld.rs'. The code defines a function 'dobra' that takes a vector of integers and returns a new vector with each element doubled. It then calls 'dobra' with a vector [1, 2, 3] and prints the result. The console shows an error: 'error: expected item, found keyword `let`' at line 7, column 1. The error message suggests that 'let' cannot be used for global variables and recommends using 'static' or 'const' instead. A note at the bottom of the error message points to the Rust reference for items.

```
1 fn dobra(v: Vec<i32>) -> Vec<i32> {
2
3   v.iter().map(|x| x * 2).collect()
4 }
5
6
7 let v = vec![1, 2, 3];
8
9 let d = dobra(v);
10
11 println!("{:?} {:?}", v, d);
```

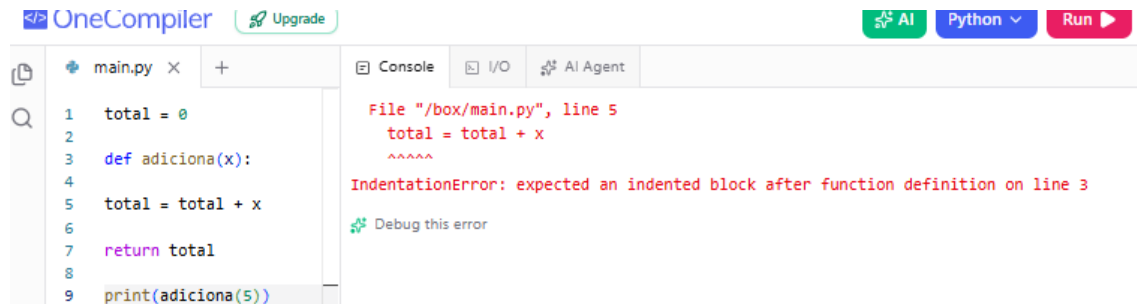
error: expected item, found keyword `let`
--> HelloWorld.rs:7:1
|
7 | let v = vec![1, 2, 3];
| ^^^
|
| `let` cannot be used for global variables
| help: consider using `static` or `const` instead of `let`
|
= note: for a full list of items that can appear in modules, see <<https://doc.rust-lang.org/reference/items.html>>

error: aborting due to 1 previous error

B: Conceito: movimento (move) e propriedade (ownership) do Rust. Ao passar v para dobra, a função assume a propriedade do vetor.

6 • Python

A:



The screenshot shows a Python IDE with a file named 'main.py'. The code defines a function 'adiciona' that takes a parameter 'x' and returns the sum of 'total' and 'x'. It then calls 'adiciona' with the value 5 and prints the result. The console shows an error: 'IndentationError: expected an indented block after function definition on line 3'. The error message points to line 3, where the function body is not properly indented.

```
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5
6
7 return total
8
9 print(adiciona(5))
```

File "/box/main.py", line 5
total = total + x
^^^^
IndentationError: expected an indented block after function definition on line 3

Debug this error

B: Conceito: escopo de variáveis. Para alterar a variável global, seria necessário usar global total.